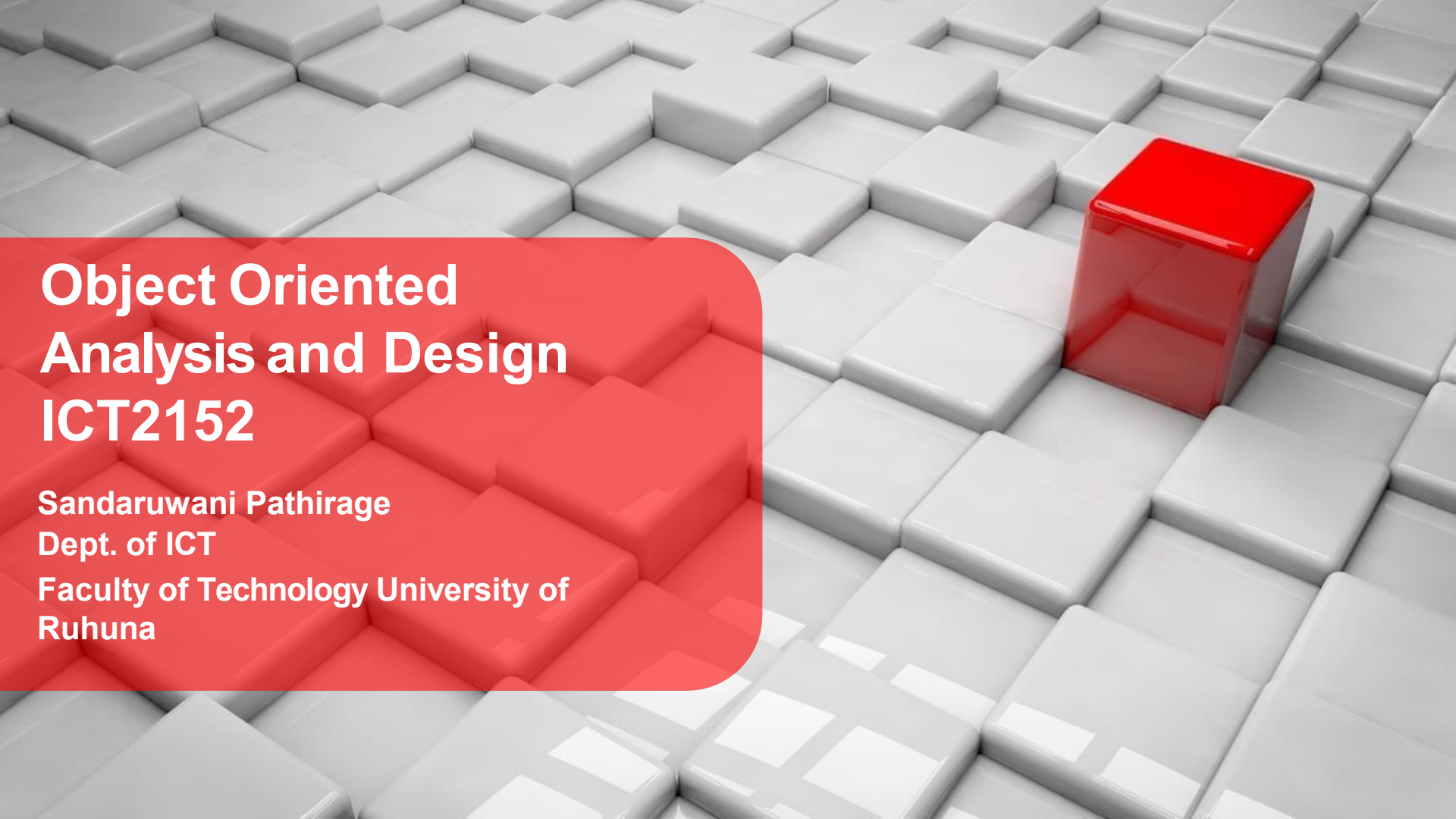


Object Oriented Analysis and Design ICT2152

Sandaruwani Pathirage
Dept. of ICT
Faculty of Technology University of Ruhuna

LECTURE 02:

Object-Oriented Languages



Objectives

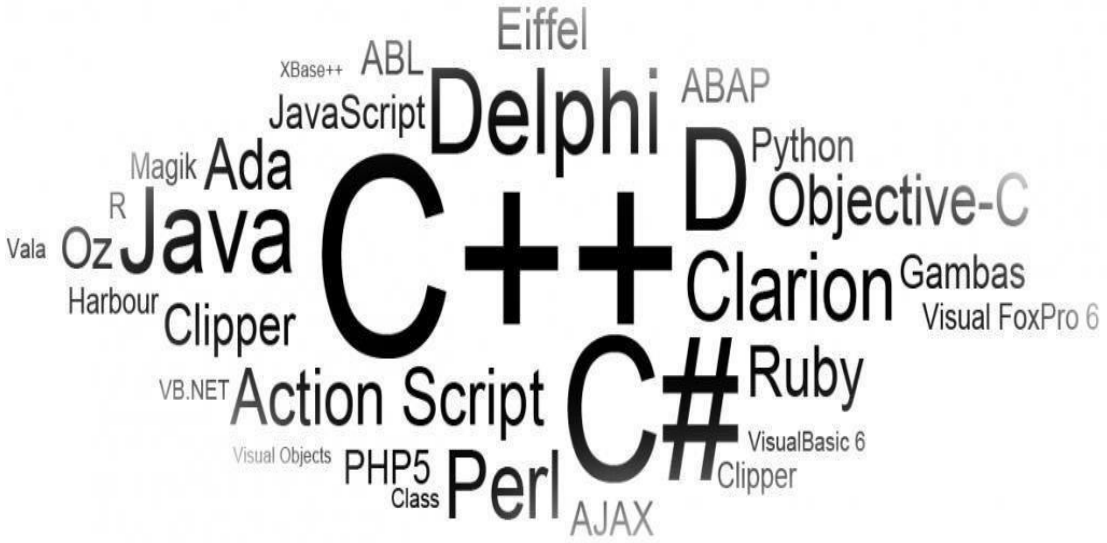
After successful completion of this lecture, students should be able to:

- Identify the object-oriented languages and the features of them.
- Explain the limitations of object orientation.



Introduction

- A number of OOP languages are available today with some differences between their capabilities and the extent to which they in accordance with to the OO paradigm.



Some Features of Object-Oriented Programming



Strong typing

- The degree of discipline that a language enforces on the programmer when declaring variables.
- "**Every data value and object that is used must belong to an appropriate type for its context**".
- Most modern languages are strongly typed.

```
int x = "Hello"; // Compile-time error in Java
```

- Prevents type mismatch errors
- Reduces runtime bugs

Some Features of Object-Oriented Programming



```java
class Vehicle {
    void start() {
        System.out.println("Vehicle starting");
    }
}

class Car extends Vehicle {
    void start() {
        System.out.println("Car starting");
    }
}

class Dog {
    void bark() {
        System.out.println("Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new Vehicle();     ✅ Vehicle variable holds Vehicle object
        v1.start();                     Output: Vehicle starting

        Vehicle v2 = new Car();         ✅ Vehicle variable can hold Car object (subclass)
        v2.start();                     Output: Car starting

        Vehicle v3 = new Dog();         ❌ Compile-time error: Dog is unrelated to Vehicle
    }
}
```

Some Features of Object-Oriented Programming



Static/ Dynamic typing

- Static typed: Check types at compile time.
- Dynamic type : Check types at run time.
- Some languages offer a hybrid approach.

Examples of Languages:

- Static: Java, C++, C#
- Dynamic: Python, Ruby, JavaScript
- Hybrid: TypeScript, Kotlin

Some Features of Object-Oriented Programming



Static/ Dynamic typing

Typing	Advantages	Disadvantages
Static	Errors caught at compile time, better performance, easier IDE support	Less flexible, verbose code
Dynamic	Flexible, faster to write, less verbose	Errors occur at runtime, harder to debug, slower performance

Some Features of Object-Oriented Programming



Static/ Dynamic typing

Static Typed (Java) – Verbose

```java
String name = "Alice";       // Must declare type
int age = 20;                // Must declare type
Person p = new Person(name, age); // Must declare exact type
```

Dynamic Typed (Python) – Less Verbose

```python
name = "Alice"       # No type declaration needed
age = 20
p = Person(name, age)
```

Garbage Collection

- Important for memory management in systems that create & delete many objects during their execution.
- Objects are removed from memory when they are deleted (otherwise the system may run out of memory in which to execute).

.



Example problem without GC:

```
for(int i = 0; i < 100000; i++) {
    Person p = new Person("Name" + i); // many objects created
    // Without GC, old objects remain in memory → may run out of memory
}
```

Types of Garbage Objects in Java

.

Nullified Objects

```
Person p = new Person("Alice");
p = null;  // "Alice" object is now garbage
```



Out of Scope Objects

```
public void createPerson() {
    Person p = new Person("Bob");
} // p goes out of scope → "Bob" object is garbage
```

Types of Garbage Objects in Java

.

Unreachable Objects via Reassignment

```
Person p1 = new Person("Alice");
Person p2 = new Person("Bob");


p1 = p2;  // "Alice" object is now unreachable → garbage
```





Standardized class libraries

- A standardized class library is a collection of predefined classes and interfaces provided by the language so programmers can reuse tested code instead of writing everything from scratch.
- Key Features:
  - Cross-platform compatibility
  - Pre-tested and reliable

Dynamic Loading

- Dynamic loading is the ability of a program to load classes or libraries into memory at runtime, rather than at compile time.

Limitations of Object Orientation



Applications Which are Not Ideally Suited to OO Development

1. **Systems that are strongly database oriented**
   - Systems which have a record based structure of data that is appropriate to a Relational Database Management System (RDBMS).
   - Also that their main processing requirement center on the storage and retrieval of the data.



Applications Which are Not Ideally Suited to OO Development



2. **Applications that are strongly algorithmic in their operation**
   - Some scientific applications that involve large & complex calculations may be neither desirable nor feasible to split the calculation down into smaller parts.
   - If such a system is implemented by OOP there might be more objects and they will be very complex.
   - Ex: Satellite orbit calculations

Exercise

In what scenarios might Object-Oriented (OO) development not be the best choice, and what are the limitations or mismatches with the nature of the problem being solved that contribute to this?



THANK YOU!